

Dokumentasjon:

1. Opprettet database og tabeller:

```
MariaDB [(none)]> CREATE DATABASE skole_fravaer_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE skole_fravaer_db;
Database changed
MariaDB [skole_fravaer_db]> CREATE TABLE IF NOT EXISTS klasser (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   navn VARCHAR(50) NOT NULL
  -> );
Query OK, 0 rows affected (0.082 sec)

MariaDB [skole_fravaer_db]>
MariaDB [skole_fravaer_db]> CREATE TABLE IF NOT EXISTS elever (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   fornavn VARCHAR(50) NOT NULL,
  ->   etternavn VARCHAR(50) NOT NULL,
  ->   klasse_id INT,
  ->   FOREIGN KEY (klasse_id) REFERENCES klasser(id)
  -> );
Query OK, 0 rows affected (0.054 sec)

MariaDB [skole_fravaer_db]>
MariaDB [skole_fravaer_db]> INSERT INTO klasser (navn)
  -> VALUES
  -> ('2ITA'),
  -> ('2ITB');
Query OK, 2 rows affected (0.011 sec)
Records: 2  Duplicates: 0  Warnings: 0

MariaDB [skole_fravaer_db]>
MariaDB [skole_fravaer_db]> INSERT INTO elever (fornavn, etternavn, klasse_id)
  -> VALUES
  -> ('Ali', 'Hansen', 1),
  -> ('Sara', 'Olsen', 1),
  -> ('Jonas', 'Berg', 2);
Query OK, 3 rows affected (0.004 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [skole_fravaer_db]>
```

```

MariaDB [skole_fravaer_db]> CREATE TABLE fag (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   navn VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.063 sec)

MariaDB [skole_fravaer_db]> CREATE TABLE fravaer (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   elev_id INT NOT NULL,
  ->   fag_id INT NOT NULL,
  ->   dato DATE NOT NULL,
  ->   status VARCHAR(50) NOT NULL,
  ->   kommentar TEXT,
  ->   FOREIGN KEY (elev_id) REFERENCES elever(id),
  ->   FOREIGN KEY (fag_id) REFERENCES fag(id)
  -> );
Query OK, 0 rows affected (0.038 sec)

MariaDB [skole_fravaer_db]> INSERT INTO fag (navn) VALUES('Informasjonstekno
logi'),('Matematikk'),('Norsk'),('Engelsk'),('Naturfag');
Query OK, 5 rows affected (0.013 sec)
Records: 5  Duplicates: 0  Warnings: 0

```

```

USE skole_fravaer_db;

CREATE TABLE fag_sporsmal (
  id INT AUTO_INCREMENT PRIMARY KEY,
  navn VARCHAR(100),
  epost VARCHAR(150),
  navn_hash VARCHAR(255),
  epost_hash VARCHAR(255),
  sporsmal TEXT NOT NULL,
  svar TEXT,
  status VARCHAR(30) DEFAULT 'ny',
  er_slettet BOOLEAN DEFAULT FALSE,
  opprettet DATETIME DEFAULT CURRENT_TIMESTAMP
);

USE skole_fravaer_db;

CREATE TABLE IF NOT EXISTS brukere (
  id INT AUTO_INCREMENT PRIMARY KEY,
  brukernavn VARCHAR(50) NOT NULL UNIQUE,
  passord_hash VARCHAR(255) NOT NULL,
  rolle VARCHAR(20) DEFAULT 'bruker'
);

```

Prosjektet bruker databasen skole_fravaer_db i MariaDB. Databasen brukes til å lagre elever, klasser, fag, fravær, brukere og spørsmål fra FAQ-siden. Flask-applikasjonen kobler seg til databasen og bruker SQL-spørringer for å hente, legge til, oppdatere og slette data.

Tabelene:

1. Tabellen **klasser** lagrer klassene som finnes i systemet. Hver klasse har en egen ID og et klassenavn. id er primærnøkkel og gjør at hver klasse får en unik ID. navn lagrer navnet på klassen. Denne tabellen brukes sammen med elever, slik at hver elev kan kobles til en klasse.
2. Tabellen **elever** lagrer informasjon om elevene i systemet. Hver elev har ID, fornavn, etternavn og en kobling til en klasse. klasse_id er en fremmednøkkel som peker til id i tabellen klasser. Dette gjør at databasen vet hvilken klasse hver elev går i. Tabellen brukes på elevlisten, elevdetaljer-siden og når man registrerer fravær.
3. Tabellen **fag** lagrer fagene som kan brukes når man registrerer fravær. Eksempler på fag kan være Informasjonsteknologi, Matematikk, Norsk og Engelsk. Denne tabellen gjør at fraværet kan kobles til et bestemt fag. Det finnes ikke en egen fag.html-side i prosjektet, men tabellen trengs fordi fravær må registreres på et fag.
4. Tabellen **fravaer** lagrer selve fraværsregistreringene. Hver registrering inneholder elev, fag, dato, status og kommentar. elev_id kobler fraværet til en elev. fag_id kobler fraværet til et fag. Dette viser bruk av relasjoner i databasen. Tabellen brukes når man registrerer fravær, viser alt fravær og ser detaljer for én elev.
5. Tabellen **brukere** lagrer brukere som kan logge inn i systemet. Den brukes til login, register og admin-rolle. brukernavn er unikt, så to brukere kan ikke ha samme brukernavn. password_hash lagrer passordet som hash, ikke som vanlig tekst. rolle bestemmer om brukeren er vanlig bruker eller admin. For eksempel kan rollen være bruker eller admin.
6. Tabellen **faq_sporsmal** lagrer spørsmål som brukere sender inn fra FAQ-siden. Tabellen lagrer navn, e-post, spørsmål, svar og status. Hvis admin anonymiserer et spørsmål, blir navn og e-post erstattet med ANONYMISERT. I tillegg lagres hash-versjoner av navn og e-post. Dette viser at prosjektet tar hensyn til personvern.

2. app.py:

- Dette er hovedfilen i Flask-prosjektet ditt. Her ligger alle routes, databasekoblingen og koden som henter, legger til, sletter og viser data fra MariaDB.

```

1  from flask import Flask, render_template
2  import mariadb
3
4  app = Flask(__name__)
5
6  def get_db_connection():
7      return mariadb.connect(
8          host="10.200.14.18",
9          user="webuser",
10         password="IMI2026*",
11         database="skole_fravaer_db"
12     )
13
14  @app.route("/")
15  def index():
16      return render_template("index.html")
17
18  @app.route("/elever")
19  def elever():
20      conn = get_db_connection()
21      cursor = conn.cursor(dictionary=True)
22
23      cursor.execute("""
24          SELECT
25              elever.id,
26              elever.fornavn,
27              elever.etternavn,
28              klasser.navn AS klasse
29          FROM elever
30          JOIN klasser ON elever.klasse_id = klasser.id
31      """)
32
33      elever_liste = cursor.fetchall()
34
35      cursor.close()
36      conn.close()
37
38      return render_template("elever.html", elever=elever_liste)
39
40
41  if __name__ == "__main__":
42      app.run(debug=True)

```

- Øverst i app.py importeres nødvendige biblioteker:

from flask import Flask, render_template, request, redirect, session

```
from werkzeug.security import generate_password_hash,  
check_password_hash
```

```
import mariadb
```

```
import hashlib
```

Flask brukes til å lage webapplikasjonen. render_template brukes til å vise HTML-filer. request brukes til å hente data fra skjemaer. redirect brukes til å sende brukeren videre til en annen side. session brukes til login. mariadb brukes til databasekobling. hashlib brukes til hashing av personinformasjon i FAQ-systemet.

- Funksjonen get_db_connection() kobler Flask-applikasjonen til MariaDB-databasen.

```
def get_db_connection():  
    return mariadb.connect(  
        host="10.200.14.18",  
        user="webuser",  
        password="*",  
        database="skole_fravaer_db",  
    )
```

Denne funksjonen brukes hver gang appen skal hente eller lagre data i databasen. Dette gjør koden mer ryddig, fordi man slipper å skrive databasekoblingen på nytt i hver route.

- Funksjonen hash_text() brukes til å lage hash av tekst.

```
def hash_text(text):  
  
    if text is None or text == "":  
  
        return None
```

```
    return hashlib.sha256(text.encode()).hexdigest()
```

I prosjektet brukes dette når FAQ-spørsmål anonymiseres. Navn og e-post kan da gjøres om til hash, slik at personinformasjon ikke vises som vanlig tekst.

- Funksjonen `er_admin()` sjekker om brukeren har rollen admin.

`def er_admin():`

`return session.get("rolle") == "admin"`

Når brukeren logger inn, lagres rollen i session. Hvis rollen er admin, får brukeren tilgang til admin-sidene. Hvis ikke, sendes brukeren tilbake til dashboard.

- **Forside-route**

`@app.route("/")`

`def index():`

`return render_template("index.html")`

Denne ruten viser forsiden til prosjektet. Den åpnes når brukeren går til `/`.

- **`/register`** brukes til å registrere nye brukere.

Når brukeren sender inn skjemaet, henter Flask brukernavn og passord.

Passordet blir hashet med `generate_password_hash()` før det lagres i databasen.

Dette er viktig fordi passord ikke bør lagres som vanlig tekst.

- `/logout` brukes til å logge ut.

`session.clear()`

Denne linjen tømmer session. Da fjernes informasjonen om hvem som er logget inn.

- `/dashboard` viser enkel statistikk.

Ruten teller antall elever, antall fravær og antall fag med SQL:

`SELECT COUNT(*) AS antall_elever FROM elever`

Dette viser hvordan man kan bruke SQL til å hente statistikk fra databasen.

- `/elever` viser alle elever.

Ruten bruker JOIN mellom elever og klasser, slik at elevens klasse vises sammen med navnet.

FROM elever

JOIN klasser ON elever.klasse_id = klasser.id

- **/legg-til-elev** brukes til å legge til nye elever.

Hvis brukeren åpner siden med GET, vises skjemaet. Hvis brukeren sender inn skjemaet med POST, lagres eleven i databasen.

- **/slett-elev/<int:id>** brukes til å slette en elev.

Først slettes fraværet som hører til eleven. Deretter slettes selve eleven. Dette gjøres fordi fravær-tabellen har en kobling til elev-tabellen.

- **/elev/<int:id>** viser detaljer for én elev.

Ruten henter først informasjon om eleven. Deretter henter den fravær som er koblet til samme elev-ID.

- **/registrer-fravaer** brukes til å registrere fravær.

Siden henter både elever og fag fra databasen, slik at brukeren kan velge riktig elev og fag i skjemaet. Når skjemaet sendes inn, lagres fraværet i databasen.

- **/fravaer** viser alt registrert fravær.

Denne ruten bruker JOIN for å vise elevnavn og fagnavn i stedet for bare ID-er. Det gjør dataene lettere å forstå for brukeren.

- **/admin** viser admin-siden.

Denne siden er beskyttet med admin-sjekk:

if not er_admin():

return redirect("/dashboard")

Det betyr at vanlige brukere ikke får tilgang til admin-siden.

- **/faq** viser FAQ-siden og håndterer innsending av spørsmål.

Hvis brukeren sender inn et spørsmål, lagres navn, e-post og spørsmål i faq_sporsmal-tabellen. Hvis siden åpnes vanlig, henter Flask besvarte spørsmål og viser dem på FAQ-siden.

- **/admin/faq** viser alle FAQ-spørsmål for admin.

Admin kan se spørsmål, navn, e-post, svar og status. Denne siden er også beskyttet slik at bare admin kan åpne den.

- **/admin/faq/svar/<int:id>** brukes når admin svarer på et spørsmål.

Svaret lagres i databasen, og status endres til besvart. Etterpå kan spørsmålet og svaret vises på vanlig FAQ-side.

- **/admin/faq/slett/<int:id>** brukes til å anonymisere personinformasjon.

Navn og e-post byttes ut med ANONYMISERT. Samtidig lagres hash-versjoner av navn og e-post, og status settes til slettet.

Dette viser at prosjektet har en enkel løsning for personvern.

➤ Templates

- templates-mappen inneholder alle HTML-filene som Flask bruker for å vise nettsider. Flask bruker Jinja2, som gjør at HTML-filene kan vise data fra Python og databasen.

3. Elever.html

elever.html viser alle elever i systemet. Elevene vises i en tabell med fornavn, etternavn og klasse. Denne siden bruker data som kommer fra en SQL-spørring i app.py. Tabellen viser

også linker for å se detaljer om en elev eller slette en elev.

```
1  <!DOCTYPE html>
2  <html lang="no">
3  <head>
4      <meta charset="UTF-8">
5      <title>Elever</title>
6      <link rel="stylesheet" href="/static/style.css">
7  </head>
8  <body>
9
10     <h1>Elevliste</h1>
11
12     <table>
13         <tr>
14             <th>ID</th>
15             <th>Fornavn</th>
16             <th>Etternavn</th>
17             <th>Klasse</th>
18         </tr>
19
20         {% for elev in elever %}
21         <tr>
22             <td>{{ elev.id }}</td>
23             <td>{{ elev.fornavn }}</td>
24             <td>{{ elev.etternavn }}</td>
25             <td>{{ elev.klasse }}</td>
26         </tr>
27         {% endfor %}
28     </table>
29
30     <br>
31
32     <a href="/">Tilbake til forsiden</a>
33
34 </body>
35 </html>
```

4. Index.html

index.html er forsiden til prosjektet. Den gir en kort introduksjon til skole-fraværssystemet. Siden har knapper videre til viktige deler av systemet, for eksempel dashboard og login. Den

fungerer som startpunkt for brukeren.

```
1 <!DOCTYPE html>
2 <html lang="no">
3 <head>
4   <meta charset="UTF-8">
5   <title>Skole fraværssystem</title>
6   <link rel="stylesheet" href="/static/style.css">
7 </head>
8 <body>
9
10  <h1>Skole fraværssystem</h1>
11
12  <p>Dette er starten på mitt SQL-prosjekt.</p>
13  <p>Systemet viser elever og hvilken klasse de går i.</p>
14
15  <a href="/elever">Se elever</a>
16
17 </body>
18 </html>
```

5. base.html

- base.html er hovedmalen i prosjektet. De andre HTML-filene bygger på denne filen.

Den inneholder felles struktur som navbar, footer, CSS-link og JavaScript-link. Dette gjør at man slipper å skrive samme navbar og footer på hver eneste side. I base.html brukes også Jinja til å vise forskjellige linker basert på om brukeren er logget inn eller ikke. Hvis brukeren er admin, vises admin-linkene

```
1 <!DOCTYPE html>
2 <html lang="no">
3 <head>
4   <meta charset="UTF-8">
5   <title>{% block title %}Skole fraværssystem{% endblock %}</title>
6   <link rel="stylesheet" href="/static/style.css">
7 </head>
8 <body>
9
10  <nav>
11    <h2>Skole fraværssystem</h2>
12
13    <div>
14      <a href="/">Hjem</a>
15      <a href="/elever">Elever</a>
16      <a href="/legg-til-elev">Legg til elev</a>
17      <a href="/registrer-fravaer">Registrer fravær</a>
18      <a href="/fravaer">Fravær</a>
19      <a href="/faq">FAQ</a>
20    </div>
21  </nav>
22
23  <main>
24    {% block content %}
25    {% endblock %}
26  </main>
27
28  <footer>
29    <p>Laget av Stefanos Gkiokas</p>
30  </footer>
31
32  <script src="/static/script.js"></script>
33 </body>
34 </html>
```

6. legg_til_elev.html

- legg_til_elev.html inneholder et skjema for å legge til en ny elev. Brukeren fyller inn fornavn, etternavn og velger klasse. Når skjemaet sendes inn, lagres eleven i elever tabellen i MariaDB.

```
1  {% extends "base.html" %}
2
3  {% block title %}Legg til elev{% endblock %}
4
5  {% block content %}
6
7  <h1>Legg til ny elev</h1>
8
9  <form method="POST">
10     <label>Fornavn:</label>
11     <input type="text" name="fornavn" required>
12
13     <label>Etternavn:</label>
14     <input type="text" name="etternavn" required>
15
16     <label>Klasse:</label>
17     <select name="klasse_id" required>
18         {% for klasse in klasser %}
19             <option value="{{ klasse.id }}">{{ klasse.navn }}</option>
20         {% endfor %}
21     </select>
22
23     <button type="submit">Lagre elev</button>
24 </form>
25
26 {% endblock %}
```

7. faq.html

- faq.html er FAQ-siden for vanlige brukere. Siden forklarer hvordan systemet brukes. Brukere kan også sende inn spørsmål med navn, e-post og spørsmål. Spørsmålet lagres i databasen og kan senere besvares av admin.

```
1  {% extends "base.html" %}
2
3  {% block title %}FAQ{% endblock %}
4
5  {% block content %}
6
7  <h1>FAQ</h1>
8
9  <div class="faq-box">
10     <h3>Hva er dette systemet?</h3>
11     <p>
12         Dette er et skole-fraværssystem som kan brukes til å registrere elever og fravær.
13     </p>
14 </div>
15
16 <div class="faq-box">
17     <h3>Hvilken teknologi bruker prosjektet?</h3>
18     <p>
19         Prosjektet bruker Flask som backend, MariaDB som database, og HTML/CSS/JavaScript som frontend.
20     </p>
21 </div>
22
23 <div class="faq-box">
24     <h3>Hvordan lagres dataene?</h3>
25     <p>
26         Dataene lagres i en SQL-database. Elever, klasser, fag og fravær ligger i egne tabeller.
27     </p>
28 </div>
29
```

```
29
30 <div class="faq-box">
31     <h3>Hvilke SQL-funksjoner viser prosjektet?</h3>
32     <p>
33         Prosjektet viser CREATE TABLE, INSERT, SELECT, DELETE, FOREIGN KEY og JOIN.
34     </p>
35 </div>
36
37 <div class="faq-box">
38     <h3>Hva kan forbedres videre?</h3>
39     <p>
40         Systemet kan forbedres med innlogging, lærerroller, søkefunksjon og fraværspersent.
41     </p>
42 </div>
43
44 {% endblock %}
```

8. elev_detaljer.html

- elev_detaljer.html viser informasjon om én bestemt elev. Siden viser elevens navn, klasse og fraværsregistreringer. Dette viser hvordan Flask kan hente data basert på en bestemt ID i URL-en.

```
1  {% extends "base.html" %}
2
3  {% block title %}Elev detaljer{% endblock %}
4
5  {% block content %}
6
7  {% if elev %}
8
9  <h1>{{ elev.fornavn }} {{ elev.etternavn }}</h1>
10
11 <div class="card">
12     <h3>Elevinformasjon</h3>
13     <p><strong>ID:</strong> {{ elev.id }}</p>
14     <p><strong>Klasse:</strong> {{ elev.klasse }}</p>
15 </div>
16
17 <h2>Fravær for denne eleven</h2>
18
19 <table>
20     <tr>
21         <th>Fag</th>
22         <th>Dato</th>
23         <th>Status</th>
24         <th>Kommentar</th>
25     </tr>
```

```
26
27     {% for f in fravaer %}
28     <tr>
29         <td>{{ f.fag }}</td>
30         <td>{{ f.dato }}</td>
31         <td>{{ f.status }}</td>
32         <td>{{ f.kommentar }}</td>
33     </tr>
34     {% endfor %}
35 </table>
36
37 {% else %}
38
39 <h1>Elev ikke funnet</h1>
40 <p>Denne eleven finnes ikke i databasen.</p>
41
42 {% endif %}
43
44 <br>
45 <a class="btn" href="/elever">Tilbake til elever</a>
46
47 {% endblock %}
```

9. registrer_fravaer.html

- registrer_fravaer.html brukes til å registrere nytt fravær. Siden har et skjema der brukeren velger elev, fag, dato, status og kommentar. Når skjemaet sendes inn, lagres fraværet i fravaer-tabellen.

```
1  {% extends "base.html" %}
2
3  {% block title %}Registrer fravær{% endblock %}
4
5  {% block content %}
6
7  <h1>Registrer fravær</h1>
8
9  <form method="POST">
10     <label>Velg elev:</label>
11     <select name="elev_id" required>
12         {% for elev in elever %}
13             <option value="{{ elev.id }}">
14                 {{ elev.fornavn }} {{ elev.etternavn }} - {{ elev.klasse }}
15             </option>
16         {% endfor %}
17     </select>
18
19     <label>Velg fag:</label>
20     <select name="fag_id" required>
21         {% for f in fag %}
22             <option value="{{ f.id }}">{{ f.navn }}</option>
23         {% endfor %}
24     </select>
25
```

```
25
26     <label>Dato:</label>
27     <input type="date" name="dato" required>
28
29     <label>Status:</label>
30     <select name="status" required>
31         <option value="Fravær">Fravær</option>
32         <option value="For sent">For sent</option>
33         <option value="Dokumentert">Dokumentert</option>
34     </select>
35
36     <label>Kommentar:</label>
37     <textarea name="kommentar" placeholder="Skriv eventuell kommentar"></textarea>
38
39     <button type="submit">Registrer fravær</button>
40 </form>
41
42 {% endblock %}
```


10. fravaer.html

- fravaer.html viser alt registrert fravær i systemet. Dataene vises i en tabell med elevnavn, fag, dato, status og kommentar. Denne siden bruker JOIN i SQL for å hente informasjon fra både fravaer, elever og fag.

```
1  {% extends "base.html" %}
2
3  {% block title %}Fravær{% endblock %}
4
5  {% block content %}
6
7  <h1>Registrert fravær</h1>
8
9  <a class="btn" href="/registrer-fravaer">Registrer nytt fravær</a>
10
11 <table>
12   <tr>
13     <th>ID</th>
14     <th>Elev</th>
15     <th>Fag</th>
16     <th>Dato</th>
17     <th>Status</th>
18     <th>Kommentar</th>
19   </tr>
20
21   {% for f in fravaer %}
22     <tr>
23       <td>{{ f.id }}</td>
24       <td>{{ f.fornavn }} {{ f.etternavn }}</td>
25       <td>{{ f.fag }}</td>
26       <td>{{ f.dato }}</td>
27       <td>{{ f.status }}</td>
28       <td>{{ f.kommentar }}</td>
29     </tr>
30   {% endfor %}
31 </table>
32
33 {% endblock %}
```

11. dashboard.html

- dashboard.html viser enkel statistikk fra databasen. Den viser for eksempel antall elever, antall fraværsregistreringer og antall fag. Denne siden viser at Flask kan hente tall fra MariaDB og vise dem på nettsiden.

```
1  {% extends "base.html" %}
2
3  {% block title %}Dashboard{% endblock %}
4
5  {% block content %}
6
7  <h1>Dashboard</h1>
8
9  <p>Her får du en enkel oversikt over dataene i systemet.</p>
10
11  <section class="cards">
12      <div class="card">
13          <h3>Antall elever</h3>
14          <p class="big-number">{{ antall_elever }}</p>
15      </div>
16
17      <div class="card">
18          <h3>Antall fravær</h3>
19          <p class="big-number">{{ antall_fravaer }}</p>
20      </div>
21
22      <div class="card">
23          <h3>Antall fag</h3>
24          <p class="big-number">{{ antall_fag }}</p>
25      </div>
26  </section>
27
28  {% endblock %}
```

12. login.html

login.html brukes til innlogging. Brukeren skriver inn brukernavn og passord. Flask sjekker om brukeren finnes i databasen, og om passordet stemmer med passord-hashen. Hvis innloggingen er riktig, lagres brukeren i en session.

```
1  {% extends "base.html" %}
2
3  {% block title %}Logg inn{% endblock %}
4
5  {% block content %}
6
7  <h1>Logg inn</h1>
8
9  <p>Logg inn for å bruke skole-fraværssystemet.</p>
10
11  {% if error %}
12      <div class="error-box">
13          {{ error }}
14      </div>
15  {% endif %}
16
17  <form method="POST">
18      <label>Brukernavn:</label>
19      <input type="text" name="brukernavn" required>
20
21      <label>Passord:</label>
22      <input type="password" name="password" required>
23
24      <button type="submit">Logg inn</button>
25  </form>
26
27  <p>
28      Har du ikke bruker?
29      <a href="/register">Registrer deg her</a>
30  </p>
31
32  {% endblock %}
```

13. register.html

- register.html brukes for å lage en ny bruker. Brukeren skriver inn brukernavn og passord. Når skjemaet sendes inn, blir passordet hashet før det lagres i databasen. Dette gjør at passordet ikke lagres som vanlig tekst.

```
1  {% extends "base.html" %}
2
3  {% block title %}Registrer bruker{% endblock %}
4
5  {% block content %}
6
7      <h1>Registrer bruker</h1>
8
9      <p>Lag en ny bruker for å få tilgang til systemet.</p>
10
11     {% if error %}
12         <div class="error-box">
13             {{ error }}
14         </div>
15     {% endif %}
16
17     <form method="POST">
18         <label>Brukernavn:</label>
19         <input type="text" name="brukernavn" required>
20
21         <label>Passord:</label>
22         <input type="password" name="passord" required>
23
24         <button type="submit">Registrer</button>
25     </form>
26
27     <p>
28         Har du allerede bruker?
29         <a href="/login">Logg inn her</a>
30     </p>
31
32     {% endblock %}
```

14. admin_faq.html

- admin_faq.html er en admin-side for innsendte FAQ-spørsmål. Her kan admin se spørsmål som brukere har sendt inn. Admin kan svare på spørsmål eller anonymisere personinformasjonen til personen som sendte spørsmålet. Spørsmålet beholdes i databasen, men navn og e-post skjules.

```
1  {% extends "base.html" %}
2
3  {% block title %}Admin FAQ{% endblock %}
4
5  {% block content %}
6
7  <h1>Admin - FAQ spørsmål</h1>
8
9  <p>
10     Her kan du se spørsmål som brukere har sendt inn. Du kan svare på spørsmål,
11     eller anonymisere personinformasjon hvis spørsmålet skal slettes.
12 </p>
13
14 <table>
15   <tr>
16     <th>ID</th>
17     <th>Navn</th>
18     <th>E-post</th>
19     <th>Spørsmål</th>
20     <th>Svar</th>
21     <th>Status</th>
22     <th>Handling</th>
23   </tr>
24
```

```

25     {% for s in sporsmal %}
26     <tr>
27         <td>{{ s.id }}</td>
28         <td>{{ s.navn }}</td>
29         <td>{{ s.epost }}</td>
30         <td>{{ s.sporsmal }}</td>
31         <td>
32             {% if s.svar %}
33                 {{ s.svar }}
34             {% else %}
35                 Ikke besvart
36             {% endif %}
37         </td>
38         <td>{{ s.status }}</td>
39         <td>
40             {% if not s.er_slettet %}
41                 <form method="POST" action="/admin/faq/svar/{{ s.id }}">
42                     <textarea name="svar" placeholder="Skriv svar her" required></textarea>
43                     <button type="submit">Svar</button>
44                 </form>
45
46                 <a class="delete-btn" href="/admin/faq/slett/{{ s.id }}" onclick="return confirmDelete();">
47                     Anonymiser
48                 </a>
49             {% else %}
50                 Personinfo anonymisert
51             {% endif %}
52         </td>
53     </tr>
54     {% endfor %}
55 </table>

```

➤ Static:

1. script.js

- script.js inneholder enkel JavaScript. I prosjektet brukes den til bekreftelse før sletting eller anonymisering:

```

1  function confirmDelete() {
2      return confirm("Er du sikker på at du vil slette denne eleven?");
3  }

```

- Dette gjør at brukeren må bekrefte før en viktig handling skjer. Det gjør systemet litt tryggere å bruke, fordi man ikke sletter eller anonymiserer noe ved et uhell.

2. style.css

- style.css bestemmer hvordan nettsiden ser ut. Filen inneholder styling for navbar, forside, knapper, skjemaer, tabeller, kort, FAQ-bokser, feilmeldinger og footer. Den gjør også nettsiden responsiv, slik at den fungerer på både mobil, PC og større skjermer.

```
1  * {
2    |   box-sizing: border-box;
3  }
4
5  html {
6    |   scroll-behavior: smooth;
7  }
8
9  body {
10   |   margin: 0;
11   |   font-family: Arial, sans-serif;
12   |   background-color: #f4f6f8;
13   |   color: #222;
14 }
15
16 /* Navbar */
17 nav {
18   |   background-color: #1e293b;
19   |   color: white;
20   |   padding: 16px 5%;
21   |   display: flex;
22   |   justify-content: space-between;
23   |   align-items: center;
24   |   gap: 20px;
25 }
26
27 nav h2 {
28   |   margin: 0;
29   |   font-size: clamp(20px, 2vw, 30px);
30   |   white-space: nowrap;
31 }
```

```
33   nav div {
34     display: flex;
35     gap: 12px;
36     flex-wrap: wrap;
37     justify-content: flex-end;
38   }
39
40   nav a {
41     color: white;
42     text-decoration: none;
43     font-weight: bold;
44     padding: 8px 10px;
45     border-radius: 6px;
46     font-size: clamp(14px, 1.2vw, 17px);
47   }
48
49   nav a:hover {
50     background-color: #334155;
51   }
52
53   /* Hovedinnhold */
54   main {
55     width: min(1200px, 90%);
56     margin: 0 auto;
57     padding: 40px 0;
58     min-height: 80vh;
59   }
60
```



```
61 footer {
62     background-color: #1e293b;
63     color: white;
64     text-align: center;
65     padding: 15px;
66 }
67
68 h1 {
69     margin-top: 0;
70     font-size: clamp(28px, 4vw, 48px);
71 }
72
73 h3 {
74     font-size: clamp(20px, 2vw, 26px);
75 }
76
77 p {
78     font-size: clamp(16px, 1.5vw, 19px);
79     line-height: 1.6;
80 }
81
```

```
82  /* Forside */
83  .hero {
84      background-color: white;
85      padding: clamp(25px, 4vw, 50px);
86      border-radius: 14px;
87      max-width: 900px;
88      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
89  }
90
91  .button-group {
92      margin-top: 20px;
93      display: flex;
94      gap: 12px;
95      flex-wrap: wrap;
96  }
97
98  .btn {
99      display: inline-block;
100     background-color: #2563eb;
101     color: white;
102     padding: 11px 18px;
103     text-decoration: none;
104     border-radius: 8px;
105     font-weight: bold;
106     font-size: 16px;
107     border: none;
108     cursor: pointer;
109 }
110
111 .btn:hover {
112     background-color: #1d4ed8;
113 }
114
```

```
114
115 .green {
116   background-color: #16a34a;
117 }
118
119 .green:hover {
120   background-color: #15803d;
121 }
122
123 /* Cards */
124 .cards {
125   display: grid;
126   grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
127   gap: 22px;
128   margin-top: 25px;
129 }
130
131 .card {
132   background-color: white;
133   padding: 22px;
134   border-radius: 12px;
135   border-left: 5px solid #2563eb;
136   box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
137 }
138
139 .card h3 {
140   margin-top: 0;
141 }
142
```

```
143  /* Tabeller */
144  ✓ table {
145      width: 100%;
146      border-collapse: collapse;
147      background-color: ■ white;
148      margin-top: 20px;
149      border-radius: 10px;
150      overflow: hidden;
151  }
152
153  ✓ th, td {
154      border: 1px solid ■ #d1d5db;
155      padding: 12px;
156      text-align: left;
157      font-size: 15px;
158  }
159
160  ✓ th {
161      background-color: ■ #2563eb;
162      color: ■ white;
163  }
164
```

```
165  /* Skjema */
166  form {
167      background-color: white;
168      padding: clamp(20px, 3vw, 35px);
169      width: min(500px, 100%);
170      border-radius: 12px;
171      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
172  }
173
174  label {
175      display: block;
176      margin-top: 15px;
177      font-weight: bold;
178  }
179
180  input, select, textarea {
181      width: 100%;
182      padding: 11px;
183      margin-top: 6px;
184      border: 1px solid #cbd5e1;
185      border-radius: 7px;
186      font-size: 16px;
187  }
188
189  textarea {
190      min-height: 90px;
191      resize: vertical;
192  }
193
```

```
193
194 button {
195     margin-top: 18px;
196     background-color: #16a34a;
197     color: white;
198     border: none;
199     padding: 11px 18px;
200     border-radius: 8px;
201     cursor: pointer;
202     font-weight: bold;
203     font-size: 16px;
204 }
205
206 button:hover {
207     background-color: #15803d;
208 }
209
210 /* Slett-knapp */
211 .delete-btn {
212     background-color: #dc2626;
213     color: white;
214     padding: 7px 12px;
215     border-radius: 6px;
216     text-decoration: none;
217     font-weight: bold;
218 }
219
220 .delete-btn:hover {
221     background-color: #b91c1c;
222 }
223
```

```
224  /* FAQ */
225  .faq-box {
226      background-color: white;
227      padding: 22px;
228      border-radius: 12px;
229      margin-bottom: 15px;
230      border-left: 5px solid #2563eb;
231      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
232  }
233
234  .faq-box h3 {
235      margin-top: 0;
236  }
237
238  .big-number {
239      font-size: 42px;
240      font-weight: bold;
241      color: #2563eb;
242  }
243
244  .small-btn {
245      padding: 7px 12px;
246      font-size: 14px;
247  }
248
```

```
248
249  /* Vanlige små skjermer */
250  @media (max-width: 800px) {
251      nav {
252          flex-direction: column;
253          align-items: flex-start;
254      }
255
256      nav div {
257          justify-content: flex-start;
258      }
259
260      nav a {
261          background-color: #334155;
262      }
263
264      main {
265          width: 92%;
266          padding: 25px 0;
267      }
268
269      .hero {
270          padding: 25px;
271      }
272
273      .button-group {
274          flex-direction: column;
275      }
276
277      .btn {
278          width: 100%;
279          text-align: center;
280      }
281
```



```
282     table {
283         display: block;
284         overflow-x: auto;
285         white-space: nowrap;
286     }
287 }
288
289 /* Ekstra små mobiler */
290 @media (max-width: 480px) {
291     nav {
292         padding: 14px 20px;
293     }
294
295     nav div {
296         width: 100%;
297         flex-direction: column;
298     }
299
300     nav a {
301         width: 100%;
302         text-align: center;
303     }
304
305     th, td {
306         padding: 9px;
307         font-size: 14px;
308     }
309
310     footer {
311         font-size: 14px;
312     }
313 }
```

```

314
315  /* Store skjermer / projektor */
316  @media (min-width: 1400px) {
317      main {
318          width: min(1300px, 85%);
319      }
320
321      .hero {
322          max-width: 1000px;
323      }
324
325      th, td {
326          font-size: 17px;
327          padding: 15px;
328      }
329
330      .btn, button {
331          font-size: 18px;
332          padding: 13px 22px;
333      }
334  }

```

Kort oppsummering

Prosjektet er bygget opp med en tydelig struktur. app.py styrer backend, routes, databasekobling, login og admin-funksjoner. templates inneholder HTML-sidene som brukeren ser. static inneholder design og JavaScript. MariaDB-databasen lagrer all viktig informasjon i tabeller med relasjoner.

Denne strukturen viser hvordan en fullstack webapplikasjon fungerer:

Bruker → HTML-side → Flask route → MariaDB → Flask → HTML-side

Dette viser både frontend, backend, database, sikkerhet og drift i samme prosjekt.

